Aperture — Low-Level Design

Module: Search

Version: v1.0

Module Owner: Discovery Platform

Dependencies:

- Metadata Module
- Users Module
- AI Module (future)
- OpenSearch (future)
- Redis

---

1. Overview

The Search module is responsible for enabling users to efficiently discover content across Aperture.

Unlike the Metadata module, which owns content, the Search module owns **retrieval**.

Its purpose is not to store data, but to provide the fastest and most relevant way to locate it.

The module should remain independent from the underlying search technology, allowing Aperture to evolve from PostgreSQL full-text search to OpenSearch and eventually semantic vector search without changing the public API.

---

2. Design Goals

The Search module should:

- Return relevant results quickly
- Support multiple searchable entity types
- Scale independently from the transactional database
- Rank results intelligently
- Support future semantic search
- Minimize database load
- Provide a consistent search experience across the application

---

3. Module Ownership

Owns

- Search requests
- Query parsing
- Ranking
- Filtering
- Autocomplete
- Search analytics
- Search index synchronization

Reads

- Metadata
- User preferences
- Trending statistics
- Popularity metrics

Never Modifies

- Movies
- TV Shows
- Reviews
- User Profiles
- Lists

---

4. Responsibilities

The Search module owns:

Search

- Movies
- TV Shows
- People
- Users
- Lists
- Collections

Filtering

Examples:

- Genre
- Year
- Runtime
- Rating
- Streaming provider
- Language
- Country

Sorting

Examples:

- Relevance
- Popularity
- Release date
- Rating
- Alphabetical

Suggestions

- Autocomplete
- Recent searches (future)
- Trending searches
- Popular queries

---

5. Search Evolution

Search intentionally evolves through three stages.

Phase 1

PostgreSQL Full-Text Search

Supports:

- title lookup
- people lookup

- basic filtering

Operational complexity remains low.

Phase 2

OpenSearch

Adds:

- fuzzy matching
- autocomplete
- typo tolerance
- faceted search
- weighted ranking

Phase 3

Semantic Search

Adds:

- natural language queries
- intent recognition
- embedding similarity
- emotional discovery

This progression keeps the architecture simple while providing a clear growth path.

---

6. High-Level Workflow

User Search

↓

Validate Request

↓

Parse Query

↓

Determine Search Type

↓

Retrieve Results

↓

Apply Ranking

↓

Apply Filters

↓

Paginate

↓

Return Response

---

7. Query Processing

Every query passes through several stages.

Validation

Reject malformed requests.

Normalization

Normalize:

- whitespace
- casing
- punctuation

Parsing

Identify:

- search terms
- filters
- sort options

Execution

Execute against the current search backend.

Ranking

Order results using the ranking strategy.

Response Formatting

Return a consistent response regardless of search backend.

---

8. Ranking Strategy

Search ranking should combine multiple signals.

Examples include:

- Text relevance
- Popularity
- Community rating
- Recency
- Exact title match
- User language
- Trending score

No single signal should dominate all searches.

The weighting strategy should remain configurable rather than hardcoded.

---

9. Search Index

The search index is considered a derived data source.

It should never become the application's source of truth.

Metadata remains authoritative.

The search index is rebuilt or updated from Metadata whenever content changes.

---

10. Index Synchronization

Triggers include:

- New movie
- Updated metadata
- New TV show
- Title change
- Artwork updates
- New keywords

Synchronization should occur asynchronously to avoid slowing user-facing operations.

---

11. Autocomplete

Autocomplete should prioritize speed over completeness.

Candidate sources:

- Popular titles
- Frequently searched content
- Exact prefix matches

Future enhancements:

- Personalized suggestions
- Search history
- Recently viewed titles

---

12. Filters

Filters should remain modular.

Adding a new filter should not require rewriting search logic.

Examples:

- Genre
- Runtime
- Language
- Streaming provider
- Decade
- Certification

Filters should compose cleanly with one another.

---

13. Performance Considerations

Potential bottlenecks:

- Broad queries
- High-traffic autocomplete
- Complex filter combinations

Mitigation strategies:

- Dedicated search index
- Result caching
- Pagination
- Query timeouts
- Background index updates

The search experience should remain responsive even under heavy load.

---

14. Caching Strategy

Candidate cache entries:

- Trending searches
- Popular autocomplete suggestions
- Frequently searched titles
- Search configuration

Avoid caching highly personalized results until personalization becomes a significant performance concern.

---

15. Security Considerations

Search endpoints are public but should still enforce:

- Rate limiting
- Input validation
- Query length limits
- Abuse detection

Administrative search operations should require authorization.

---

16. Error Handling

Representative failures:

- Invalid query
- Unsupported filter
- Search backend unavailable
- Timeout
- Partial index update

Fallback behavior:

If the dedicated search backend is unavailable, the system should gracefully fall back to PostgreSQL full-text search where feasible.

Availability is preferred over perfect relevance.

---

17. Testing Strategy

Unit Tests

- Query parsing
- Ranking logic
- Filter composition
- Validation

Integration Tests

- Search index synchronization
- Metadata integration
- Cache invalidation

API Tests

- Search endpoint
- Autocomplete
- Filter combinations
- Pagination

Performance Tests

Measure:

- Query latency
- Index update time
- Autocomplete response time
- Large result pagination

---

18. Future Evolution

Potential enhancements:

- Semantic reranking
- Natural language search
- Personalized search
- Voice search
- Image-based search
- Search analytics dashboard
- AI-generated search summaries

The module should support these capabilities without requiring changes to consuming clients.

---

19. Interview Discussion Points

Be prepared to explain:

- Why separate Search from Metadata?

- Why evolve from PostgreSQL to OpenSearch instead of starting there?
- Why is the search index not the source of truth?
- How do you synchronize search indexes efficiently?
- How would you rank results beyond simple text matching?
- How would you scale autocomplete for millions of users?

---

20. Summary

The Search module is designed as an independent retrieval platform rather than a thin database wrapper.

By isolating search concerns from metadata ownership and introducing progressively more sophisticated search technologies, Aperture can deliver a fast and relevant discovery experience while maintaining architectural flexibility and operational simplicity.